

上海财经大学

信息管理与工程学院

网络流与时空网络流

优化理论与算法

王震

2026

本节结构：从瓶颈到时空网络

网络流是优化中最重要的结构之一。本节不只讲模型，更强调它和整数规划之间的关系。

模块	要回答的问题
最大流与最小割	一个网络最多能通过多少流量，瓶颈在哪里
最小费用流	满足供需时，如何以最低成本运输或调度
整数性	为什么很多网络流 LP 不加整数约束也能得到整数解
时空网络流	如何把“地点 + 时间”的调度问题转化为网络流
与整数规划的关系	什么时候网络流仍然好解，什么时候会变成困难 MIP

全局符号表

本节统一采用以下记号，后续模型只在局部案例中补充少量专用符号。

符号	含义
$G = (N, A)$	有向网络； N 为节点集合， A 为弧集合
$(i, j) \in A$	从节点 i 指向节点 j 的有向弧
x_{ij}, u_{ij}, c_{ij}	弧 (i, j) 上的流量、容量和单位费用
s, t, F	最大流中的源点、汇点和总流量
b_i	节点 i 的净供给量； $b_i > 0$ 为供给， $b_i < 0$ 为需求
N^{ST}, A^{ST}	时空节点集合与时空弧集合，节点通常写作 (i, t)

为什么网络流重要

很多看起来不同的问题，本质上都是“资源在网络上移动”。

场景	节点表示	弧表示
物流运输	仓库、转运点、门店	可运输路线
库存计划	不同时期的库存状态	生产、持有、满足需求
车辆调度	任务或时空位置	接续、等待、空驶
人员排班	时间与工作状态	上班、休息、转移

核心思想

网络流模型把复杂决策转化为：流量沿着弧移动，并在节点处满足守恒关系。

图的基本语言：点和边

一个图由节点和边组成，通常记为

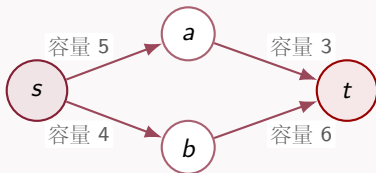
$$G = (N, A).$$

记号	含义
N	节点集合，也叫顶点集合，例如仓库、城市、任务、时间状态
A	弧集合，也叫有向边集合，例如道路、运输连接、任务接续关系
$(i, j) \in A$	一条从节点 i 指向节点 j 的有向弧

为什么用有向图

网络流通常关心资源从哪里流向哪里，因此弧的方向很重要。 (i, j) 和 (j, i) 是两条不同的弧。

图中的路径和容量



概念	含义
路径	一串首尾相接的有向弧，例如 $s \rightarrow a \rightarrow t$
容量 u_{ij}	弧 (i, j) 上最多允许通过的流量
流量 x_{ij}	实际选择让多少资源通过弧 (i, j)

网络建模关系：从业务到数学

网络流建模的关键，是把业务对象翻译成节点、弧、流量和守恒关系。

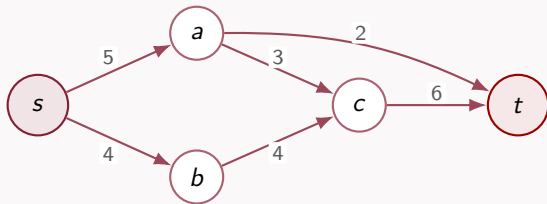
建模元素	对应的数学含义
节点	资源所在的位置、时间、任务状态或库存状态
弧	资源允许发生的动作，例如运输、等待、生产、服务、接续
流量	有多少资源选择某个动作，通常记为 x_{ij} 或 x_a
容量	某个动作最多能被使用多少，通常记为 u_{ij} 或 u_a
守恒	资源进入一个节点后，必须通过某些弧离开或被需求消耗

网络建模的核心

先定义节点，再定义允许的弧，最后用流守恒描述资源如何在网络中移动。

第一部分：最大流问题

考虑一个从源点 s 到汇点 t 的运输网络。每条弧 (i,j) 有容量 u_{ij} ，表示最多能通过多少流量。



弧上数字表示容量

问题

在不超过每条弧容量的前提下，最多可以从 s 向 t 送出多少流量？

为什么要研究最大流

最大流问题回答的是网络的通行能力。

应用场景	最大流回答的问题
------	----------

物流网络	从仓库到市场最多能运多少货
------	---------------

通信网络	从源端到终端最多能传多少数据
------	----------------

救援疏散	单位时间最多能疏散多少人
------	--------------

生产系统	从原料到成品的最大通过能力是多少
------	------------------

重要性

最大流不仅给出一个可行运输方案，还帮助识别整个系统的最大处理能力；这类“能力上限”问题是物流、通信、生产和应急系统中的基本问题。

最大流模型

令 x_{ij} 表示弧 (i,j) 上的流量, F 表示从源点 s 送到汇点 t 的总流量。

$$\begin{aligned} \max \quad & F \\ \text{s.t.} \quad & \sum_{j:(i,j) \in A} x_{ij} - \sum_{j:(j,i) \in A} x_{ji} = \begin{cases} F, & i = s, \\ 0, & i \in N \setminus \{s, t\}, \\ -F, & i = t, \end{cases} \quad i \in N, \\ & 0 \leq x_{ij} \leq u_{ij}, \quad (i,j) \in A. \end{aligned}$$

模型解释

源点净流出为 F , 汇点净流出为 $-F$, 中间节点净流出为 0。容量约束保证每条弧上的流量不超过其通行能力。

最大流模型的性质：流分解

最大流模型虽然使用弧变量 x_{ij} ，但任意可行流都可以理解为若干条 s - t 路径流和若干个环流的叠加。

流分解性质

给定一个可行流 x ，存在若干条 s - t 路径 P 及对应流量 $\alpha_P \geq 0$ ，再加上一些环流，使得弧流量可以由这些路径和环上的流量相加得到。

结论	含义
流量可以走多条路径	最大流不是寻找一条最宽路径，而是把网络容量综合利用起来
环流不增加 s - t 流值	若目标只关心最大流值，纯环流可以去掉而不降低目标值

最大流模型的性质：LP 但常有整数解

最大流模型是线性规划，但它具有很强的网络结构。

整数性性质

若所有容量 u_{ij} 都是整数，则最大流问题存在一个整数最优解。

普通 LP	最优解可能是分数
-------	----------

最大流 LP	在整数容量下，可以得到整数最优流
--------	------------------

建模意义	即使流量代表车辆数、人员数、订单数，也通常不需要额外加入整数约束
------	----------------------------------

直观原因

节点-弧关联矩阵具有特殊结构；后面的增广路算法也会给出一个构造性解释。

最大流模型的性质：对偶会导向瓶颈

从优化角度看，最大流是一个“最大化通行能力”的 LP。它的对偶问题要寻找一个最小上界。

原问题视角	对偶视角
安排弧上的流量	给网络中的容量限制定价
最大化从 s 到 t 的总流量	寻找阻断 s 到 t 的最小瓶颈
可行流说明“能做到多少”	对偶上界说明“最多不超过多少”

过渡

最大流模型的对偶结构最终会表现为最小 $s - t$ 割。接下来先定义割，再证明它与最大流之间的精确关系。

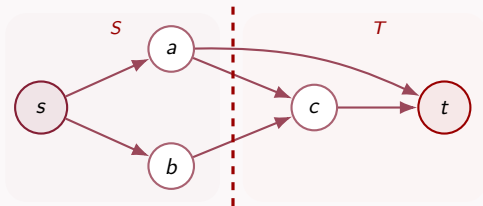
什么是 $s - t$ 割

一个 $s - t$ 割把节点集合分成两部分：

$$(S, T), \quad s \in S, \quad t \in T, \quad S \cup T = N.$$

割容量定义为

$$u(S, T) = \sum_{i \in S, j \in T} u_{ij}.$$



直观含义

割表示一组把源点和汇点分开的瓶颈弧。所有从 s 到 t 的流都必须跨过任意一个 $s - t$ 割。

最小割问题：瓶颈在哪里

割容量越小，说明这组跨越 S 和 T 的弧越可能成为系统瓶颈。

$$\min u(S, T) = \sum_{i \in S, j \in T} u_{ij}$$

$$s \in S, \quad t \in T, \quad S \cup T = N, \quad S \cap T = \emptyset.$$

最小割的意义

最小割寻找的是把 s 和 t 分开所需要切断的最小总容量。它告诉我们网络中最关键的瓶颈位置。

术语提醒

本节讲的是最小 $s - t$ 割。通常所说的 **Max-Cut**（最大割）是另一个问题：它最大化跨割边权重，和最大流-最小割定理不是同一个问题。

最小割的 0-1 建模

最小 $s - t$ 割也可以写成 0-1 线性整数规划。令

$$y_i = \begin{cases} 1, & i \in S, \\ 0, & i \in T, \end{cases} \quad z_{ij} = \begin{cases} 1, & i \in S, j \in T, \\ 0, & \text{否则}. \end{cases}$$

$$\begin{aligned} \min \quad & \sum_{(i,j) \in A} u_{ij} z_{ij} \\ \text{s.t.} \quad & z_{ij} \geq y_i - y_j, \quad (i,j) \in A, \\ & y_s = 1, \quad y_t = 0, \\ & y_i \in \{0, 1\}, \quad i \in N, \\ & z_{ij} \in \{0, 1\}, \quad (i,j) \in A. \end{aligned}$$

建模逻辑

若 $i \in S, j \in T$, 则 $y_i - y_j = 1$, 约束强制 $z_{ij} = 1$; 否则 $y_i - y_j \leq 0$, 目标函数会使不必要的 z_{ij} 取 0。

最小割模型的解释

变量 y_i 决定节点 i 在割的哪一侧；变量 z_{ij} 判断弧 (i,j) 是否从 S 指向 T 。

约束	含义
$y_s = 1, y_t = 0$	源点和汇点必须被分在两侧
$z_{ij} \geq y_i - y_j$	若 $i \in S, j \in T$ ，则必须令 $z_{ij} = 1$
$\min \sum u_{ij} z_{ij}$	最小化所有跨割弧的总容量

整数规划视角

这个模型看起来是 0-1 整数规划，但最小 $s-t$ 割具有特殊网络结构，可以通过最大流算法或等价的 LP 对偶关系高效求解。

为什么任意流不超过任意割

对任意 $s - t$ 割 (S, T) ，从 S 流向 T 的净流量等于总流值。

因此任意可行流值 F 都满足：

$$F = \sum_{i \in S, j \in T} x_{ij} - \sum_{i \in T, j \in S} x_{ij} \leq \sum_{i \in S, j \in T} u_{ij} = u(S, T).$$

命题：割给出最大流的上界

任意可行流的流值不超过任意 $s - t$ 割的容量。

任意可行流值 $\leq$ 任意割容量。

优化视角

可行流给出一个“能做到多少”的下界；割给出一个“不可能超过多少”的上界。

最大流-最小割定理

定理：最大流-最小割

在任意有向容量网络中，从 s 到 t 的最大流值等于最小 $s - t$ 割容量：

$$\max\{\text{流值}\} = \min\{u(S, T) : (S, T) \text{ 是 } s - t \text{ 割}\}.$$

对象	作用
最大流	给出一个可行运输方案
最小割	给出一个证明最优的瓶颈证书
二者相等	说明没有任何方案能超过当前流量

证明路线一：最大流对偶与割松弛

最大流 LP 的对偶可以等价写成下面的形式：

$$\begin{aligned} \min \quad & \sum_{(i,j) \in A} u_{ij} z_{ij} \\ \text{s.t.} \quad & z_{ij} \geq y_i - y_j, & (i,j) \in A, \\ & y_s = 1, \quad y_t = 0, \\ & 0 \leq y_i \leq 1, & i \in N, \\ & z_{ij} \geq 0, & (i,j) \in A. \end{aligned}$$

与最小割 0-1 模型的关系

这正是前面最小割 0-1 模型去掉整数约束后的 LP 松弛。变量 y_i 表示节点在 S 侧的“程度”，变量 z_{ij} 表示弧 (i,j) 被割开的“程度”。

对偶推导第一步：改写最大流 LP

为方便求对偶，令 $b_s = 1, b_t = -1, b_i = 0$ ($i \neq s, t$)。最大流模型可写为

$$\begin{aligned} \max \quad & F \\ \text{s.t.} \quad & \sum_{j:(i,j) \in A} x_{ij} - \sum_{j:(j,i) \in A} x_{ji} - b_i F = 0, & i \in N, \\ & x_{ij} \leq u_{ij}, & (i,j) \in A, \\ & x_{ij} \geq 0, & (i,j) \in A. \end{aligned}$$

对偶推导中将 F 视为自由变量；最优最大流值自然非负。

约束	对偶变量
节点平衡等式	π_i 自由变量, $i \in N$
容量约束 $x_{ij} \leq u_{ij}$	$z_{ij} \geq 0$, $(i,j) \in A$

对偶推导第二步：整理成割松弛

按线性规划对偶规则，对偶问题为

$$\begin{aligned} \min \quad & \sum_{(i,j) \in A} u_{ij} z_{ij} \\ \text{s.t.} \quad & z_{ij} + \pi_i - \pi_j \geq 0, & (i,j) \in A, \\ & - \sum_{i \in N} b_i \pi_i = 1, \\ & z_{ij} \geq 0, & (i,j) \in A. \end{aligned}$$

其中

$$\sum_{i \in N} b_i \pi_i = \pi_s - \pi_t, \quad \text{所以} \quad \pi_t - \pi_s = 1.$$

由于只出现势差，可以平移令 $\pi_t = 0$ 。再令

$$y_i = \pi_t - \pi_i,$$

就得到

$$z_{ij} \geq y_i - y_j, \quad y_s = 1, \quad y_t = 0.$$

将 y_i 限制在区间 $[0, 1]$ 不会变差：小于 0 的值可截为 0，大于 1 的值可截为 1，约束 $z_{ij} \geq y_i - y_j$ 和目标值不会变大。

割是对偶松弛的特殊可行解

若给定一个真正的 $s - t$ 割 (S, T) , 令

$$y_i = \begin{cases} 1, & i \in S, \\ 0, & i \in T, \end{cases} \quad z_{ij} = \begin{cases} 1, & i \in S, j \in T, \\ 0, & \text{否则}. \end{cases}$$

则它满足对偶 LP 的全部约束, 且目标值为

$$\sum_{(i,j) \in A} u_{ij} z_{ij} = \sum_{\substack{i \in S \\ j \in T}} u_{ij} = u(S, T).$$

结论

最小割是对偶 LP 的一个整数可行解。因此对偶 LP 最优值不超过最小割容量。

阈值取整：从分数解到一个割

取任意对偶 LP 可行解 (y, z) 。其中 $0 \leq y_i \leq 1$ ，可以把 y_i 看成节点 i 位于源点侧 S 的程度。
对任意阈值 $\theta \in (0, 1]$ ，定义

$$S_\theta = \{i \in N : y_i \geq \theta\}, \quad T_\theta = N \setminus S_\theta.$$

条件	结论
$y_s = 1$	对所有 $\theta \in (0, 1]$ ，都有 $s \in S_\theta$
$y_t = 0$	对所有 $\theta \in (0, 1]$ ，都有 $t \in T_\theta$

结论

每一个阈值 $\theta \in (0, 1]$ 都给出一个真正的 $s - t$ 割 (S_θ, T_θ) 。

一条弧什么时候会被阈值割切到

固定一条弧 (i, j) 。它在阈值割 (S_θ, T_θ) 中被计入割容量，当且仅当

$$i \in S_\theta, \quad j \in T_\theta.$$

也就是

$$y_i \geq \theta > y_j.$$

因此，能让弧 (i, j) 被切到的阈值 θ 只可能落在区间

$$(y_j, y_i].$$

情形	被切到的阈值长度
$y_i > y_j$	长度为 $y_i - y_j$
$y_i \leq y_j$	不会从 S_θ 指向 T_θ ，长度为 0

所以弧 (i, j) 被切到的阈值长度为

$$\max\{y_i - y_j, 0\}.$$

由 LP 约束 $z_{ij} \geq y_i - y_j$ 和 $z_{ij} \geq 0$ ，可得

$$\max\{y_i - y_j, 0\} \leq z_{ij}.$$

阈值取整：平均容量不超过 **LP** 目标值

把 θ 在 $(0, 1]$ 上均匀变化。上一页说明：弧 (i, j) 被阈值割切到的比例不超过 z_{ij} 。

因此阈值割容量的平均值满足

$$\int_0^1 u(S_\theta, T_\theta) d\theta \leq \sum_{(i,j) \in A} u_{ij} z_{ij}.$$

平均值推出存在性

如果所有阈值割容量都大于 $\sum u_{ij} z_{ij}$ ，那么它们的平均值也会大于 $\sum u_{ij} z_{ij}$ ，与上式矛盾。

所以至少存在一个 θ ，使得

$$u(S_\theta, T_\theta) \leq \sum_{(i,j) \in A} u_{ij} z_{ij}.$$

阈值取整证明 LP 松弛无间隙

上一页说明：对任意对偶 LP 可行解 (y, z) ，都可以找到一个真正的割 (S_θ, T_θ) ，其容量不超过该 LP 解的目标值。因此对偶 LP 最优值不会低于最小割容量；另一方面，任意真正的割本身就是对偶 LP 的可行解。于是

$$\min_{\text{割}(S, T)} u(S, T) = \min_{\text{对偶 LP 可行}(y, z)} \sum_{(i, j) \in A} u_{ij} z_{ij}.$$

再由 LP 强对偶定理：

最大流值 = 对偶 LP 最优值 = 最小割容量.

证明路线二：残量网络证明

残量网络证明给出一个更算法化的解释。证明同样分两步：

方向	证明内容
上界方向	任意可行流值 F 都不超过任意割容量 $u(S, T)$
等号方向	若残量网络中无 s - t 路径，则由可达点集合构造割，并证明 $F = u(S, T)$

与对偶证明的关系

对偶证明强调 LP 强对偶；残量网络证明强调算法停止时如何直接得到最小割证书。

残量网络证明第一步：割容量给出上界

设 x 是任意可行流, (S, T) 是任意 $s-t$ 割。由于 $s \in S, t \in T$, 从 S 流向 T 的净流量等于总流值:

$$F = \sum_{\substack{i \in S \\ j \in T}} x_{ij} - \sum_{\substack{i \in T \\ j \in S}} x_{ij}.$$

又因为 $x_{ij} \leq u_{ij}$ 且 $x_{ij} \geq 0$, 所以

$$F \leq \sum_{\substack{i \in S \\ j \in T}} x_{ij} \leq \sum_{\substack{i \in S \\ j \in T}} u_{ij} = u(S, T).$$

结论

最大流值不可能超过最小割容量:

$$\max F \leq \min_{(S, T)} u(S, T).$$

残量网络证明第二步：由可达点构造割

设 x 是一个可行流，并且它的残量网络中不存在从 s 到 t 的路径。

定义

$$S = \{i \in N : \text{在残量网络中从 } s \text{ 可以到达 } i\}, \quad T = N \setminus S.$$

由于 $s \in S$ ，且不存在从 s 到 t 的残量路径，所以

$$t \notin S, \quad t \in T.$$

因此 (S, T) 是一个合法的 $s - t$ 割。

构造思想

残量网络中从 s 还能到达的点放在 S ；到达不了的点放在 T 。没有增广路保证 s 和 t 被这个割分开。

残量网络证明第三步：跨割弧的流量状态

考虑由残量可达集合构造出的割 (S, T) 。

弧的方向	结论
$(i, j) \in A, i \in S, j \in T$	必有 $x_{ij} = u_{ij}$ ，否则还有正向残量 $u_{ij} - x_{ij} > 0$ ，节点 j 也应可达
$(i, j) \in A, i \in T, j \in S$	必有 $x_{ij} = 0$ ，否则残量网络中存在反向弧 $j \rightarrow i$ ，节点 i 也应可达

含义

从 S 指向 T 的割边全部饱和；从 T 指向 S 的反向跨割边没有流量。

残量网络证明第四步：流值等于割容量

由流守恒，当前流值等于跨割净流量：

$$F = \sum_{\substack{i \in S \\ j \in T}} x_{ij} - \sum_{\substack{i \in T \\ j \in S}} x_{ij}.$$

根据上一页结论，

$$x_{ij} = u_{ij} \quad (i \in S, j \in T), \quad x_{ij} = 0 \quad (i \in T, j \in S).$$

因此

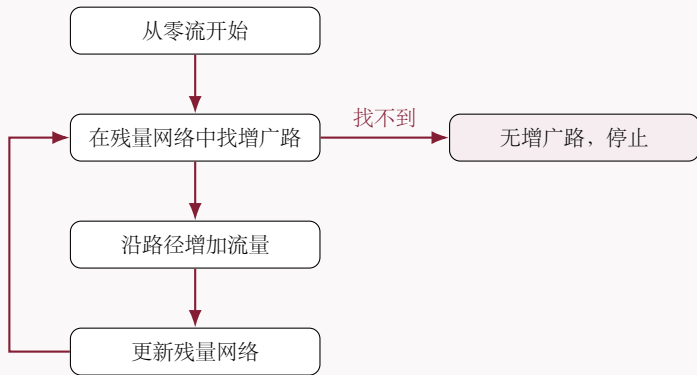
$$F = \sum_{\substack{i \in S \\ j \in T}} u_{ij} = u(S, T).$$

定理结论

因为任意流值都不超过任意割容量，而当前可行流 x 与割 (S, T) 满足 $F = u(S, T)$ ，所以 x 是最大流， (S, T) 是最小割。

增广路算法直觉

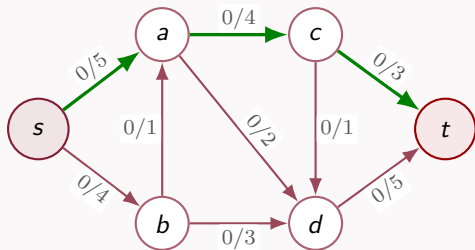
Ford-Fulkerson 方法的基本思想非常直接。



停止含义：当残量网络中找不到从 s 到 t 的路径时，可以由可达节点集合直接构造一个最小割。

增广路算法图示：第一步

用 x/u 表示“当前流量/容量”。绿色表示本轮选择的增广路，其余弧保持原始颜色，已经送出的流量由数字体现。



本轮增广

选择路径

$$s \rightarrow a \rightarrow c \rightarrow t.$$

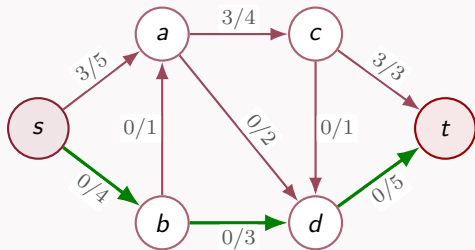
这条路径的剩余容量为

$$\min\{5, 4, 3\} = 3.$$

因此总流量增加 3。

增广路算法图示：继续增广

第一次增广后， $c \rightarrow t$ 已经满载，但网络中仍然存在新的可用路径。



第二轮

选择路径

$$s \rightarrow b \rightarrow d \rightarrow t.$$

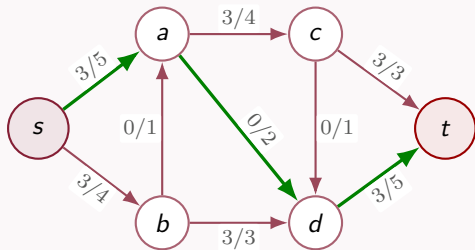
瓶颈容量为

$$\min\{4, 3, 5\} = 3.$$

总流量从 3 增加到 6。

增广路算法图示：最后一次增广

第二次增广后，仍可通过 $a \rightarrow d$ 把剩余流量送到汇点。



第三轮

选择路径

$$s \rightarrow a \rightarrow d \rightarrow t.$$

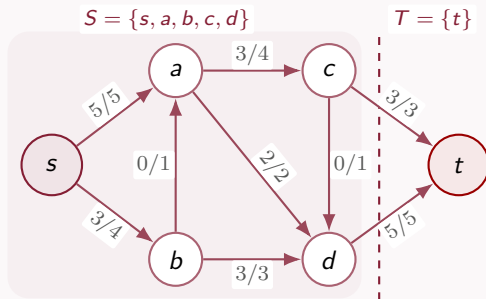
瓶颈容量为

$$\min\{2, 2, 2\} = 2.$$

总流量增加到 8。

增广路算法图示：停止与最小割

第三次增广后，虽然 $s \rightarrow b$ 仍有剩余容量，但从 s 可达的节点已经无法再连到 t 。



为什么停止

通向 t 的割边容量为

$$u_{ct} + u_{dt} = 3 + 5 = 8.$$

当前流量也是 8, 因此最大流值为 8。

这个割就是一个最优性证书：
任何流都不可能超过它的容量。

增广路算法的作用

增广路算法不仅是一种求解方法，也把最大流问题的几个核心概念连接在一起。

作用	具体含义
求最大流	通过反复寻找可用路径，逐步增加从 s 到 t 的流量
识别瓶颈	当不存在增广路时，可由残量网络中的可达点集合构造一个最小割
证明最优	当前流量等于某个割容量，因此已经达到所有可行流的上界
解释整数性	若容量为整数，每次增广量为整数，因此得到整数流

教学重点

最大流模型说明“要求什么”；增广路算法说明“如何求”；最小割说明“为什么不能更大”。

增广路算法的相关性质

性质一：无增广路即最优

若当前可行流的残量网络中不存在从 s 到 t 的路径，则该可行流是最大流。

性质二：整数容量给出整数流

若所有容量 u_{ij} 都是整数，则 Ford-Fulkerson 增广过程中每次增广量都是整数，因此可得到整数最大流。

复杂度提醒

如果每次随意选择增广路，算法效率可能受路径选择影响。Edmonds-Karp 算法规定每次选择边数最少的增广路，可以保证多项式时间复杂度 $O(|N||A|^2)$ 。

为什么随机选路也能保证最优

随机选择增广路会影响迭代次数，但不改变终止时的最优性判断。

命题：终止时一定最优

若当前流可行，且残量网络中不存在从 s 到 t 的增广路，则当前流是最大流。

1. 每次沿增广路增加瓶颈流量，仍满足容量约束和流守恒；
2. 若没有增广路，从 s 在残量网络中可达的点构成集合 S ，且 $t \notin S$ ；
3. 从 S 到 T 的正向弧已经没有剩余容量，否则还能继续走出去；
4. 因而当前流量等于割 (S, T) 的容量，达到所有流的上界。

结论：路径选择影响“快慢”，无增广路加最小割证书保证“正确”。

最大流中的整数性

命题：整数容量给出整数最大流

如果所有容量 u_{ij} 都是整数，则最大流问题存在一个整数最优解。

算法直觉

增广路算法从零流开始；如果容量是整数，每次增广的流量也是整数，因此整个过程中流量始终为整数。

最大流看起来像一个需要整数流量的问题，但在这种网络结构下，LP 松弛已经足够强，通常不需要显式写整数约束。

第二部分：最小费用流

最大流关心“最多能送多少”，最小费用流关心“满足供需时，怎样送成本最低”。

最大流	每条弧只有容量，目标是最大化从源点到汇点的总流量
-----	--------------------------

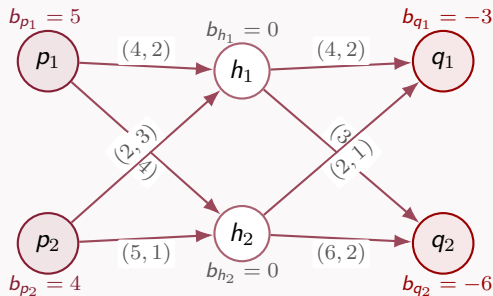
最小费用流	每条弧既有容量又有费用，目标是在满足节点供需的条件下最小化总费用
-------	----------------------------------

典型问题

多个仓库向多个门店配送，中间可以经过转运中心；每条路线有运力上限和单位运输成本，要求满足门店需求且总成本最低。

最小费用流：完整图结构

节点给出供给或需求，弧给出“容量上限 u ”和“单位费用 c ”。弧标签写作 (u, c) 。



读图方式： p_1, p_2 是供给节点， q_1, q_2 是需求节点， h_1, h_2 是中转节点；例如 $(4, 2)$ 表示容量为 4、单位费用为 2。

从图到模型：变量和约束

设 x_{ij} 表示弧 (i,j) 上运输的流量。最小费用流模型由三类信息组成。

对象	数学表达
弧流量变量	$x_{ij} \geq 0$ ，表示从节点 i 到节点 j 的流量
容量约束	$0 \leq x_{ij} \leq u_{ij}$ ，每条弧不能超过运力上限
节点平衡	流出量 - 流入量 = b_i ，供给节点净流出，需求节点净流入
费用目标	总费用为 $\sum_{(i,j) \in A} c_{ij} x_{ij}$

可行性提醒

若网络中没有外部损耗，通常需要 $\sum_{i \in N} b_i = 0$ 。总供给必须等于总需求。

最小费用流标准模型

给定有向网络 $G = (N, A)$ ，节点供需量 b_i ，弧容量 u_{ij} ，单位费用 c_{ij} 。

$$\begin{aligned} \min \quad & \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \sum_{j: (i,j) \in A} x_{ij} - \sum_{j: (j,i) \in A} x_{ji} &= b_i, \quad i \in N \\ 0 \leq x_{ij} &\leq u_{ij}, \quad (i,j) \in A. \end{aligned}$$

节点含义

$b_i > 0$ 表示供给节点， $b_i < 0$ 表示需求节点， $b_i = 0$ 表示中转节点。这个模型仍然是线性规划，但它具有特殊网络结构。

最小费用流包含哪些经典问题

最小费用流是许多网络优化模型的统一框架。

经典问题	如何看成最小费用流
最短路问题	发送 1 单位流，从起点到终点，弧费用为路径长度
运输问题	供给点直接连到需求点，弧费用为单位运输成本
指派问题	每个工人供给 1，每个任务需求 1，弧费用为分配成本
最大流问题	费用设为 0，并加入从汇点回到源点的负费用弧，可转化为费用流形式

为什么重要

它不是一个孤立模型，而是连接最短路、运输、指派、最大流和时空调度问题的基础结构。

最小费用流与整数规划

定理：最小费用流的整数性

若供需量 b_i 和容量 u_{ij} 均为整数，则最小费用流问题存在整数最优解。

这一性质来自网络流约束矩阵的特殊结构。更一般地，可以用全酉模矩阵解释。

普通整数规划	LP 松弛可能给出分数解
--------	--------------

网络流模型	很多情况下 LP 松弛天然给出整数解
-------	--------------------

教学重点

不是所有带整数含义的模型都难。有些模型结构足够好，线性规划本身就能给出整数解。

最小费用流的最优性直觉

最小费用流的核心问题不是“能不能送”，而是“当前送法还能不能更便宜”。

命题：负费用环与改进方向

若残量网络中存在总费用为负的有向环，则沿该环调整一部分流量可以降低总费用；
若不存在负费用环，则当前可行流是最优流。

最大流	找从 s 到 t 的增广路，增加总流量
-----	-------------------------

最小费用流	找能降低费用的残量路径或残量环，改进当前流的成本
-------	--------------------------

直观解释

如果存在一圈调整方案，使得流量守恒不被破坏、容量不被违反、总费用还下降，那么当前方案还不是最优。

最小费用流常用算法

最小费用流是多项式时间可解问题，但不同算法利用的结构不同。

算法思想	适合如何理解
逐次最短路算法	每次从供给端向需求端发送一批流，选择当前最便宜的可行路径
负环消去算法	如果残量网络中存在负费用环，就沿环调整流量，直到没有负费用环
网络单纯形法	把线性规划单纯形法专门化到网络结构，是工程软件中很重要的方法
费用缩放算法	通过逐步提高费用精度控制改进过程，适合大规模网络的理论分析和实现

共同点：不把问题当作一般 LP，而是利用“节点平衡 + 弧容量 + 路径/环”的网络结构。

最小费用流的研究主线

基础最小费用流已经有成熟算法。后续研究通常围绕三类问题展开。

研究方向	典型问题
更快的算法	如何在节点和弧非常多时求得更快，如何利用稀疏网络、动态更新和并行计算
更复杂的约束	加入固定启用成本、车辆数量、服务时间窗、多种货物后，模型往往变成困难的整数规划
更真实的不确定性	需求、旅行时间、成本不确定时，需要随机优化、鲁棒优化或在线算法

重要分界：单商品最小费用流通常“好解”；多商品、固定成本和时间窗等扩展常常会变成 **NP-hard** 的整数规划。

最小费用流的应用研究

许多运营管理问题都可以先抽象为最小费用流，再根据业务约束扩展。

应用领域	最小费用流视角
物流配送	商品从仓库、转运中心流向门店，费用表示运输成本或时间成本
库存与生产	不同时期之间的库存持有可看成时间弧，生产和缺货可看成不同费用弧
车辆与人员调度	任务之间的接续关系形成网络，流表示车辆或人员的工作轨迹
能源与通信网络	电力、数据或资源在网络中传输，费用表示损耗、拥塞或运行成本

和后续内容的关系：时空网络流是最小费用流的重要扩展，节点表示“地点与时间”的组合。

最小费用流：核心结论

结论	含义
统一框架	最短路、运输、指派和许多调度问题都可以看成最小费用流
线性模型	目标是费用最小，约束是节点平衡和弧容量
整数性	若 b_i, u_{ij} 为整数，最小费用流存在整数最优解
最优性判断	残量网络中不存在负费用环时，当前可行流已经最优

讲授提示

最大流强调“能送多少”，最小费用流强调“怎样送最便宜”。二者都是网络结构带来强可解性的典型例子。

从最小费用流到时空网络流

时空网络流本质上是把时间维度展开后的最小费用流。

普通最小费用流	节点表示地点或状态，弧表示资源移动
---------	-------------------

时空网络流	节点表示“地点 + 时间”，弧表示等待、运输、服务或回收
-------	------------------------------

共同结构	都使用流量变量、节点守恒、容量约束和费用目标
------	------------------------

过渡思想

只要能把业务动作放到时间线上，就可以尝试构造时空节点和时空弧，再使用网络流模型表达调度决策。

第三部分：为什么需要时空网络

普通网络只描述“地点之间能否连接”，但很多运营问题还必须描述“什么时候发生”。

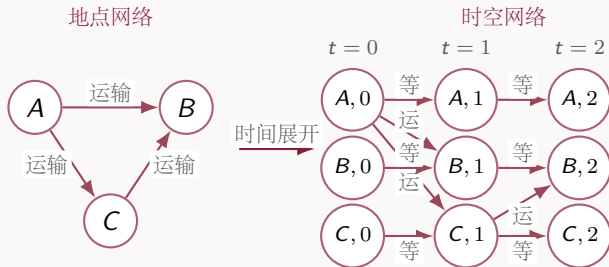
问题	普通网络能表达	还缺少什么
物流运输	仓库到门店的路线	发车时间、到达时间、车辆等待
库存计划	生产、库存、需求关系	库存在不同时期之间如何转移
车辆调度	任务之间是否可接续	任务开始/结束时间与空驶时间

核心想法

把每个地点 i 在每个时间 t 复制成一个节点 (i, t) ，于是“位置变化”和“时间推进”都可以用弧表示。

从地点网络到时空网络：图示

左边是地点网络，右边是把每个地点按时间复制后的时空网络。



读图方式：横向表示时间推进，纵向表示不同地点；一条时空路径就是资源随时间移动或等待的轨迹。

时空网络中的弧类型

时空网络的关键不是多画一些节点，而是把业务动作翻译成不同类型的弧。

弧类型	建模含义
等待弧	$(i, t) \rightarrow (i, t + 1)$ ，资源停留在地点 i ，常表示库存持有、车辆等待
运输弧	$(i, t) \rightarrow (j, t + \tau_{ij})$ ，资源从地点 i 运输到地点 j
服务弧	从服务开始节点到服务完成节点，表示执行一个订单、任务或班次
外部弧	从超级源点进入网络或从网络流向超级汇点，表示初始资源和最终回收

建模原则

先定义时空节点，再定义哪些业务动作允许发生；每一个允许动作对应一条时空弧。

时空网络建模步骤

从业务问题到时空网络模型，可以按下面五步完成。

步骤	需要确定的内容
1. 定义集合	地点集合 L 和离散时间集合 $\mathcal{T}$
2. 构造节点	对每个可用地点和时间建立节点 (i, t)
3. 添加弧	加入等待弧、运输弧、服务弧和外部弧
4. 定义变量	对每条时空弧 $a \in A^{ST}$ 定义流量 x_a
5. 写出模型	节点守恒、弧容量和费用目标 $\min \sum c_a x_a$

时空网络流: Notation

为了写出统一模型，先明确时空网络中的符号。

符号	含义
L	地点集合，例如仓库、门店、车场、任务位置
$\mathcal{T} = \{0, 1, \dots, H\}$	离散时间集合， H 是计划期长度
N^{ST}	时空节点集合，典型节点为 (i, t) ，表示时间 t 位于地点 i
A^{ST}	时空弧集合，包括等待弧、运输弧、服务弧和外部弧
x_a	弧 $a \in A^{ST}$ 上的流量，可表示货物量、车辆数、人员数
c_a, u_a, b_v	弧费用、弧容量、节点净供给量
$\delta^+(v), \delta^-(v)$	从时空节点 v 出发和进入 v 的弧集合
τ_{ij}	从地点 i 到地点 j 的运输或移动时间

注意： N^{ST} 和 A^{ST} 是人为构造出来的，构造方式决定了模型是否准确。

时空网络流通用模型

在构造好时空节点和时空弧后，模型仍然是最小费用流的形式。

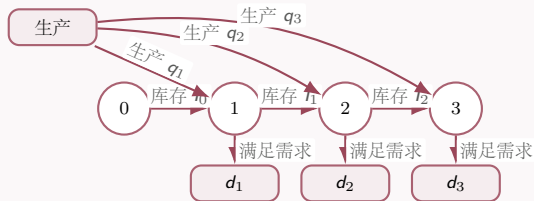
$$\begin{aligned} \min \quad & \sum_{a \in A^{ST}} c_a x_a \\ \sum_{a \in \delta^+(v)} x_a - \sum_{a \in \delta^-(v)} x_a &= b_v, \quad v \in N^{ST} \\ 0 \leq x_a &\leq u_a, \quad a \in A^{ST}. \end{aligned}$$

模型解释

节点平衡表示“资源不能凭空产生或消失”；弧容量限制库存容量、运输能力、车辆数量或服务能力；目标函数累计等待、运输、服务和外部调度成本。

案例一：库存计划的时空网络

考虑单产品、多时期库存计划。时间节点表示“期初库存状态”，生产、库存持有和需求满足都可以写成弧。



弧	决策含义
生产弧	当期生产量 q_t ，费用为生产成本
库存弧	从 t 留到 $t+1$ 的库存 i_t ，费用为持有成本
需求弧	满足当期需求，通常流量固定为 d_t

库存计划：对应的数学模型

库存平衡方程

$$I_t = I_{t-1} + q_t - d_t, \quad t = 1, \dots, H$$

就是时间网络上每个节点的流守恒。

$$\min \sum_{t=1}^H p_t q_t + \sum_{t=1}^H h_t I_t$$

$$I_t = I_{t-1} + q_t - d_t, \quad t = 1, \dots, H$$

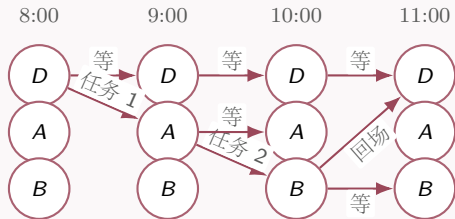
$$0 \leq q_t \leq Q_t, \quad 0 \leq I_t \leq \bar{I}_t.$$

和时空网络的对应

q_t 为生产量, I_t 为库存, d_t 为需求, p_t, h_t 为生产和持有成本, $Q_t, \bar{I}_t$ 为生产和库存上限。生产弧贡献 q_t , 库存弧贡献 I_t , 需求弧消耗 d_t 。

案例二：车辆调度的时空网络

车辆调度中，节点可以表示“某辆车在某地点某时刻可用”。运输、等待和执行任务都是弧。



读图方式

一条从车场 D 出发并最终回到 D 的时空路径，表示一辆车的完整工作轨迹。

车辆调度：建模符号与约束

设 x_a 表示使用时空弧 a 的车辆数。不同弧有不同业务含义。

符号或约束	含义
$x_a \in \{0,1\}$ 或 $x_a \geq 0$ 整数	是否有车辆执行弧 a ，或有多少辆车经过弧 a
节点流守恒	车辆到达某个时空节点后，要么等待，要么执行任务，要么返回
任务覆盖约束	每个必须服务的任务弧需要被选择一次
车辆数量约束	从车场源点出发的总流量不超过可用车辆数

何时变难

如果只是不带复杂资源的单商品时空流，通常较好求；加入任务覆盖、司机工时、充电、时间窗等约束后，往往进入整数规划或列生成框架。

网络流与整数规划：什么时候变难

单商品线性网络流通常结构很好；困难往往来自“网络结构之外”的耦合、逻辑和资源约束。

新增结构	为什么变难
固定启用成本	需要 0-1 变量 y_{ij} ，例如 $x_{ij} \leq u_{ij}y_{ij}$
多商品流	不同商品共享容量，流之间产生耦合
路径数量限制	需要控制选择多少条路径或车辆
时间窗与资源约束	路径不只要连通，还要满足容量、时间、工时等资源限制

判断标准

如果模型仍然只是单商品流守恒和弧容量，通常较好求；一旦需要选择是否启用、区分多类资源或控制整条路径，就常常进入整数规划。

与列生成的关系

许多列生成问题中的“列”，本质上是网络中的一条路径。

列生成问题	定价子问题常见形式
车辆路径	带容量和时间窗的最短路问题
机组排班	满足法规和休息要求的路径问题
生产或库存计划	时间展开网络上的最小费用路径或流问题

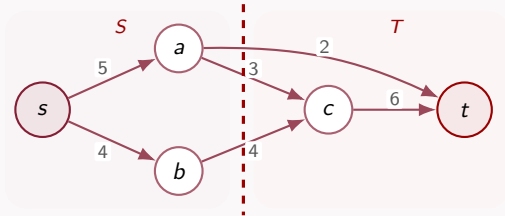
列生成中的新列 = 网络中的一条可行路径.

衔接

网络流提供了很多定价子问题的算法基础；列生成则用于处理路径或方案数量极多的情况。

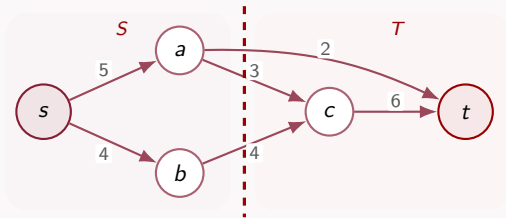
课堂练习：找出割容量

给定下图，请找出割 $S = \{s, a\}$ 、 $T = \{b, c, t\}$ 的容量。



课堂练习：找出割容量

给定下图，请找出割 $S = \{s, a\}$ 、 $T = \{b, c, t\}$ 的容量。



参考答案

从 S 指向 T 的弧为 (s, b) 、 (a, c) 、 (a, t) ，割容量为

$$4 + 3 + 2 = 9.$$

课堂练习：画出时空网络

某车辆在两个地点 A、B 之间运行，时间为 $t = 1, 2, 3$ 。车辆可以：

- ▶ 在同一地点等待一个时间单位；
- ▶ 从 A 到 B 运输，用时一个时间单位；
- ▶ 从 B 到 A 返回，用时一个时间单位。

请画出时空节点

$(A, 1), (A, 2), (A, 3), (B, 1), (B, 2), (B, 3)$

以及对应的等待弧和运输弧。

课堂练习：画出时空网络

某车辆在两个地点 A、B 之间运行，时间为 $t = 1, 2, 3$ 。车辆可以：

- ▶ 在同一地点等待一个时间单位；
- ▶ 从 A 到 B 运输，用时一个时间单位；
- ▶ 从 B 到 A 返回，用时一个时间单位。

请画出时空节点

$(A, 1), (A, 2), (A, 3), (B, 1), (B, 2), (B, 3)$

以及对应的等待弧和运输弧。

提示

等待弧连接 $(i, t) \rightarrow (i, t + 1)$ ，运输弧连接 $(A, t) \rightarrow (B, t + 1)$ 或 $(B, t) \rightarrow (A, t + 1)$ 。

本节小结

- ▶ 最大流问题刻画网络中的最大通行能力；
- ▶ 最小割给出最大流的瓶颈证书，最大流值等于最小割容量；
- ▶ 最小费用流在满足供需的前提下最小化运输或调度成本；
- ▶ 网络流模型具有特殊结构，很多情况下 LP 松弛天然存在整数最优解；
- ▶ 时空网络把地点和时间一起建模，适合库存、车辆调度和排班问题；
- ▶ 固定成本、多商品、逻辑约束和复杂资源会把网络流推向整数规划；
- ▶ 列生成中的新列常常是网络中的一条可行路径。